Lecture 3: ML-KEM (Kyber) — A Real Scheme, End to End

Parameters, Compression, CPA-Secure Encryption, and the Route to CCA Security

Dr. Faisal Aslam

Lecture Notes — Session 3a (approx. 2 hours)

Purpose of this lecture. Lecture 2 ended with a promise: a real Kyber public key would turn out to be "just an MLWE sample." This lecture cashes that promise in. We fix concrete parameters, define the compression functions real ciphertexts use, write out Kyber's core encryption scheme (KeyGen / Enc / Dec) algorithm-by-algorithm, prove — with real numbers — why decryption actually recovers the message, and then explain the one extra transformation (Fujisaki–Okamoto) that turns this into the standardized, chosen-ciphertext-secure **ML-KEM (FIPS 203)**. That last step deserves special attention: it is exactly the part of the scheme most often targeted by the physical attacks in Section 2 of the proposal.

Contents

1 Recap: What You Already Have 1

2 Concrete Parameters 1

3 Compression: Trading Precision for Size 2

4 The CPA-Secure Core: Kyber.CPAPKE 2

5 Why Decryption Works: the Noise-Cancellation Argument 3

6 A Fully Worked Example, By Hand 4

7 From CPA-Secure Encryption to a CCA-Secure KEM 4

8 Deployment Status 6

9 Summary Table 6

10 Exercises (Assign Before Lecture 4) 7

1 Recap: What You Already Have

From Lecture 2: Module-LWE says that, for a small integer k and $R_q = \mathbb{Z}_q[x]/(x^n + 1)$, the pair $(\vec{a}, b = \langle \vec{a}, \vec{s} \rangle + e)$ is computationally indistinguishable from uniform random, for secret $\vec{s} \in R_q^k$ and small error e. Today, $\vec{a}$ becomes a whole *matrix* A of ring elements (one MLWE sample per row), $\vec{s}$ becomes the actual Kyber secret key, and b's vector analogue $\vec{t} = A\vec{s} + \vec{e}$ becomes the actual Kyber public key. Nothing new needs to be invented — we are about to watch Lecture 2's abstract object become a working encryption scheme.

2 Concrete Parameters

Definition: ML-KEM Parameter Sets

All three standardized parameter sets share $n = 256$ (ring dimension) and $q = 3329$ (modulus, a prime chosen so $q \equiv 1 \pmod{256}$, which makes the NTT fast — we will not need the NTT itself, just this reason for q's odd-looking value). They differ only in the module rank k and the noise parameters η_1, η_2 (which control how "wide" the centered binomial noise distribution — Lecture 2, Section 4.5 — is):

Parameter set	k	η_1	η_2	Public-key size
ML-KEM-512	2	3	2	800 bytes
ML-KEM-768	3	2	2	1,184 bytes
ML-KEM-1024	4	2	2	1,568 bytes

Remark: Reading This Table With Lecture 2's Eyes

k is exactly the module rank from Lecture 2, Section 6 — the secret key is a length-k vector of ring elements, $\vec{s} \in R_q^k$, and the public matrix A is $k \times k$ (ring elements). Larger k means more MLWE "dimensions" to search over, hence higher security — and a bigger public key, since you're now storing k ring elements (each 256 coefficients) instead of one. This is the size/security dial promised back in Lecture 2's Figure 3.

3 Compression: Trading Precision for Size

A ciphertext, as defined so far, would consist of full elements of R_q — every one of 256 coefficients stored with full precision mod $q = 3329$ (about 12 bits each). Kyber shrinks ciphertexts by deliberately throwing away low-order bits of precision, in a controlled way.

Definition: Compression and Decompression

For an integer $d < \lceil \log_2 q \rceil$, define, for $x \in \mathbb{Z}_q$:

$$\text{Compress}_d(x) = \left\lceil \frac{2^d}{q} \cdot x \right\rfloor \bmod 2^d, \qquad \text{Decompress}_d(y) = \left\lceil \frac{q}{2^d} \cdot y \right\rfloor,$$

where $\lceil \cdot \rfloor$ denotes rounding to the nearest integer. Compression maps a $\mathbb{Z}_q$ value down to a d-bit value; decompression maps it back up to (approximately) the original value. Applied coefficient-by-coefficient, these extend to compressing/decompressing an entire ring element.

Example: Compressing and Decompressing by Hand, $q = 17$, $d = 2$

With $q = 17$ and $d = 2$ (so values get squeezed down to 2 bits $\in \{0, 1, 2, 3\}$): take $x = 9$. Then $\text{Compress}_2(9) = \lceil \frac{4}{17} \cdot 9 \rfloor = \lceil 2.12 \rfloor = 2$. Decompressing: $\text{Decompress}_2(2) = \lceil \frac{17}{4} \cdot 2 \rfloor = \lceil 8.5 \rfloor = 9$ (or 8, depending on rounding convention for the exact half-way case) — in this instance we land back exactly on 9. In general you do *not* get back the original value exactly — that's the entire point: Compress then Decompress introduces a small, bounded rounding error, in exchange for needing only $d = 2$ bits instead of $\lceil \log_2 17 \rceil = 5$ bits to store or transmit x.

Remark: Why Deliberately Add More Error?

This seems backwards — Lecture 2 spent an entire lecture explaining that noise is what makes LWE hard, and now we're adding *more* noise on purpose. The resolution: encryption already tolerates a noise budget (Section 5 below shows exactly how much). Compression consumes a small, carefully bounded slice of that budget in exchange for a large reduction in ciphertext size — as long as the total noise (compression error *plus* the original LWE error) stays under the budget, decryption still succeeds. Push d too low, though, and decryption starts failing — this is a real, quantifiable trade-off the standard's parameters were chosen to balance, and it's also precisely the kind of margin that a fault attack (Session 5) tries to push past its limit on purpose.

4 The CPA-Secure Core: Kyber.CPAPKE

We now write out the actual scheme. To keep hand-computation possible later, we describe it at the module level using Lecture 2's notation directly; A^T denotes the transpose of the matrix of ring elements, and all arithmetic is in R_q.

Algorithm 1 Kyber.CPAPKE.KeyGen

1: Generate $A \in R_q^{k \times k}$ uniformly at random (in practice: expanded deterministically from a short public seed ρ via a hash-based expander, so only ρ — not all of A — needs to be stored or sent)
2: Sample $\vec{s} \in R_q^k$, each coefficient from the small noise distribution (centered binomial, parameter η_1)
3: Sample $\vec{e} \in R_q^k$ the same way
4: Compute $\vec{t} = A\vec{s} + \vec{e} \in R_q^k$
5: **return** public key $pk = (A, \vec{t})$, secret key $sk = \vec{s}$

Algorithm 2 Kyber.CPAPKE.Enc$(pk = (A, \vec{t}),\ m \in \{0,1\}^n)$

1: Sample $\vec{r} \in R_q^k$, $\vec{e}_1 \in R_q^k$ from the small noise distribution (parameter η_1), and $e_2 \in R_q$ (parameter η_2)
2: Compute $\vec{u} = A^T\vec{r} + \vec{e}_1 \in R_q^k$
3: Encode the message: $\mathrm{Encode}(m) \in R_q$ has i-th coefficient $\lceil q/2 \rceil \cdot m_i$ (each bit becomes either 0 or "as far from 0 as possible mod q")
4: Compute $v = \vec{t}^T\vec{r} + e_2 + \mathrm{Encode}(m) \in R_q$
5: **return** ciphertext $c = \big(\mathrm{Compress}_{d_u}(\vec{u}),\ \mathrm{Compress}_{d_v}(v)\big)$

Algorithm 3 Kyber.CPAPKE.Dec$(sk = \vec{s},\ c = (c_u, c_v))$

1: Recover $\vec{u} = \mathrm{Decompress}_{d_u}(c_u)$, $v = \mathrm{Decompress}_{d_v}(c_v)$
2: Compute $m' = v - \vec{s}^T\vec{u} \in R_q$
3: **return** $\mathrm{Decode}(m')$: for each coefficient of m', output bit 1 if it is closer to $\lceil q/2 \rceil$ than to 0 (mod q), else bit 0

Remark: Recognizing Lecture 2 Inside This Algorithm

Line 4 of KeyGen, $\vec{t} = A\vec{s} + \vec{e}$, is exactly an MLWE sample, verbatim, from Lecture 2 Section 6. Encryption generates a *second*, independent MLWE-style sample ($\vec{u}$) using a fresh secret $\vec{r}$, and hides the message inside a third quantity v that mixes the public key with that same $\vec{r}$. Nothing about the shape of these equations is new — only the bookkeeping (which noise term uses which name, and the extra Encode/Compress steps) is specific to Kyber.

5 Why Decryption Works: the Noise-Cancellation Argument

Remark: The Algebra Behind Line 2 of Decryption (Ignoring Compression for a Moment)

Substitute the definitions of $\vec{u}$ and v into $m' = v - \vec{s}^T\vec{u}$:

$$\begin{aligned} m' &= \left(\vec{t}^T\vec{r} + e_2 + \text{Encode}(m)\right) - \vec{s}^T\left(A^T\vec{r} + \vec{e}_1\right) \\ &= \vec{t}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \text{Encode}(m) \\ &= (A\vec{s} + \vec{e})^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \text{Encode}(m) \qquad \text{(substituting } \vec{t} = A\vec{s} + \vec{e}\text{)} \\ &= \vec{s}^T A^T\vec{r} + \vec{e}^T\vec{r} - \vec{s}^T A^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1 + \text{Encode}(m) \\ &= \text{Encode}(m) + \underbrace{\left(\vec{e}^T\vec{r} + e_2 - \vec{s}^T\vec{e}_1\right)}_{\text{total noise, call it } \varepsilon}. \end{aligned}$$

The two copies of $\vec{s}^T A^T\vec{r}$ **cancel exactly** — this is the whole trick, and it's why the scheme is designed around A appearing on both sides (once via $\vec{t}$, once via $\vec{u}$). What's left is the message, plus a total noise term ε built entirely from small quantities ($\vec{e}, \vec{r}, e_2, \vec{s}, \vec{e}_1$ are all "small" by construction). As long as every coefficient of ε stays smaller than $q/4$ in absolute value, m''s coefficients land close enough to 0 or $\lceil q/2 \rceil$ that Decode recovers m exactly.

Remark: Decryption Failure: a Real, Quantifiable Possibility

If ε's coefficients happen to be unusually large (all the small noise terms landing far from 0 by chance, or compression eating too much of the remaining budget), decryption can output the *wrong* bit. This isn't a flaw — it's an accepted, carefully bounded probability (ML-KEM's parameters keep it astronomically small, far below 2^{-100} per ciphertext) baked into the design. It matters for your project for a specific reason: some physical attacks work by *deliberately inducing* decryption failures (via faults, or via chosen ciphertexts that sit right at the edge of the noise budget) and then use the pattern of failures to leak information about $\vec{s}$ — Session 5's material on Kyber's central-reduction leak and CCA-style attacks builds directly on this margin.

6 A Fully Worked Example, By Hand

Example: Kyber-Style Encryption/Decryption, $k = 1$, $n = 4$, $q = 17$ (Verified by Hand)

Take $R_{17} = \mathbb{Z}_{17}[x]/(x^4+1)$ and module rank $k = 1$ (so everything below is a single ring element, not a vector — this is really Ring-LWE-flavored Kyber, kept at $k = 1$ purely so the arithmetic stays hand-tractable).

KeyGen. Let $A = 3 + 5x + 2x^2 + x^3$ (public). Let $s = 1 - x$ (secret, small coefficients) and $e = 1$ (small error). Multiplying in R_{17} (using $x^4 \equiv -1$, exactly as in Lecture 2's polynomial-multiplication example):

$$A \cdot s = 4 + 2x + 14x^2 + 16x^3, \qquad t = A \cdot s + e = 5 + 2x + 14x^2 + 16x^3.$$

Public key: (A, t). Secret key: $s = 1 - x$.

Enc. Message $m = (1, 0, 1, 0)$ (four bits, one per coefficient). Encode with threshold value $9 \approx \lceil 17/2 \rceil$: $\text{Encode}(m) = 9 + 0x + 9x^2 + 0x^3$. Sample small randomness $r = x$, $e_1 = 1$, $e_2 = -1$. Compute:

$$u = A \cdot r + e_1 = 3x + 5x^2 + 2x^3 + x^4 + 1 = 0 + 3x + 5x^2 + 2x^3,$$

$$\begin{aligned} v &= t \cdot r + e_2 + \text{Encode}(m) \\ &= (1 + 5x + 2x^2 + 14x^3) + (-1) + (9 + 0x + 9x^2 + 0x^3) \\ &= 9 + 5x + 11x^2 + 14x^3. \end{aligned}$$

(We skip $\text{Compress}/\text{Decompress}$ here to keep the arithmetic focused — Section 3's example already showed compression separately.) Ciphertext: $c = (u, v)$.

Dec. Compute $s \cdot u = 2 + 3x + 2x^2 + 14x^3$, then

$$m' = v - s \cdot u = (9 - 2) + (5 - 3)x + (11 - 2)x^2 + (14 - 14)x^3 = 7 + 2x + 9x^2 + 0x^3.$$

Decode each coefficient (closer to 9 than to 0, mod 17, means bit 1): $7 \to 1$, $2 \to 0$, $9 \to 1$, $0 \to 0$. **Recovered message:** $(1, 0, 1, 0)$ **— exactly the original.** Every step above is real Kyber structure, just at toy scale; nothing was simplified away except using $k = 1$ instead of $k \in \{2, 3, 4\}$ and full 256-coefficient rings.

7 From CPA-Secure Encryption to a CCA-Secure KEM

Kyber.CPAPKE, as just described, is only secure against an attacker who never gets to submit chosen ciphertexts for decryption (a **CPA** — chosen-plaintext-attack — guarantee). Real protocols (TLS, VPNs) need more: an attacker who *can* submit arbitrary ciphertexts and observe what happens (timing, error messages, side channels) still shouldn't learn anything. That stronger notion is **CCA security**, and it's what ML-KEM actually provides — via a generic transformation, not a redesign of the algebra above.

Definition: Key Encapsulation Mechanism (KEM)

A KEM replaces "encrypt an arbitrary message" with a narrower, safer interface: $\text{Encaps}(pk)$ outputs *both* a ciphertext c *and* a fresh shared secret K; $\text{Decaps}(sk, c)$ recovers the same K. Neither party ever chooses K directly — it's generated as a by-product of encapsulation. K is then used as a symmetric key for the rest of the session (e.g., to key AES inside TLS).

Remark: The Fujisaki–Okamoto (FO) Transform, Informally

The key idea: instead of encrypting a message chosen by an attacker, **encrypt a message that only the honest sender ever knows**, and make the ciphertext itself depend on that message in a way the receiver can double-check. Concretely, ML-KEM's Encaps generates a fresh random message m, derives both the encryption randomness *and* the final shared secret K from m (via hashing), and encrypts m using Kyber.CPAPKE as above. On the receiving end, Decaps does something distinctive: it decrypts c to recover a candidate m', then **re-encrypts m' using the exact same (deterministic, derived-from-m') randomness** and checks whether it gets back the *same* ciphertext c. If yes, K is derived from m' normally. If the re-encryption check fails — meaning c wasn't a ciphertext that any honest sender would have produced — **Decaps does not error out**. Instead it returns a *different*, pseudorandom-looking K (derived from the secret key and c together, "implicit rejection"), indistinguishable to an outside observer from a legitimate key.

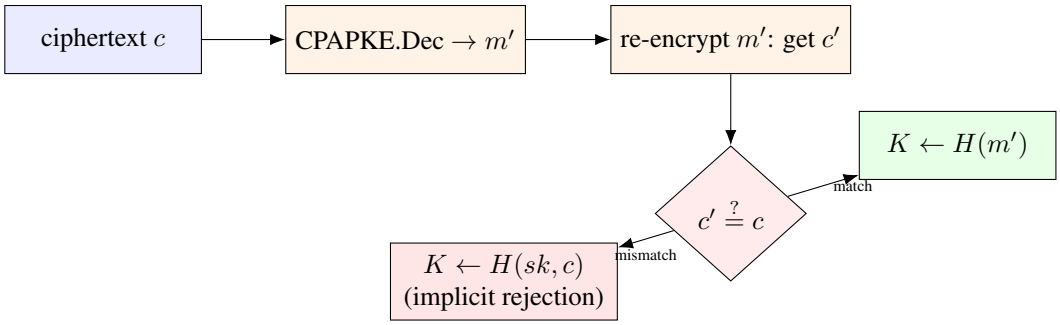

Figure 1: ML-KEM's Decaps, schematically. The re-encryption check ($c' \stackrel{?}{=} c$) is the step the Fujisaki–Okamoto transform adds on top of plain Kyber.CPAPKE decryption — and, because both branches must be computed in a way that reveals nothing about *which* branch was taken (equal time, no data-dependent memory access), it is also one of the most scrutinized few lines of code in the entire scheme from a side-channel perspective.

Looking Ahead: Why This One Step Gets an Entire Attack Literature (Preview of Session 5)

Two properties make the FO re-encryption check a magnet for physical attacks. First, it is a *comparison* whose outcome (match vs. mismatch) must never leak — if an attacker can learn, via timing or power consumption, whether a chosen malformed ciphertext passed or failed this check, they can often turn many such queries into a full key-recovery attack (this is the shape of several published Kyber side-channel results). Second, the intermediate value m' computed on line 1 (*before* the check) briefly exists in memory even for a ciphertext that will ultimately be rejected — if that intermediate computation itself leaks (e.g. through the CPAPKE decryption arithmetic touching the secret $\vec{s}$), the rejection branch doesn't help you, because the damage already happened one step earlier. Keep both of these observations in mind; Session 5 names specific published attacks that exploit exactly this structure.

8 Deployment Status

ML-KEM is not experimental. It was finalized by NIST as **FIPS 203** in August 2024, alongside ML-DSA (FIPS 204) and SLH-DSA (FIPS 205).[1] It is already deployed at scale: Chrome enabled hybrid ML-KEM by default in late 2024, Apple's iMessage uses it as part of PQ3, and the major cloud providers

[1] NIST, *Module-Lattice-Based Key-Encapsulation Mechanism Standard*, FIPS 203, August 2024.

(AWS, Google Cloud, Microsoft Azure) support PQC-enabled TLS endpoints. In these hybrid deployments, ML-KEM typically runs *alongside* a classical algorithm like X25519 rather than replacing it outright, so that security holds even if one of the two turns out to have an unforeseen weakness.

9 Summary Table

Object	**What it is, in Lecture 2's language**
Public key $(A, \vec{t})$	An MLWE sample: $\vec{t} = A\vec{s} + \vec{e}$
Secret key $\vec{s}$	The MLWE secret
Ciphertext part $\vec{u}$	A second, independent MLWE sample using fresh randomness $\vec{r}$
Ciphertext part v	Public key mixed with $\vec{r}$, plus the message, plus noise
Decryption	Noise cancels algebraically, leaving message + small residual noise
CPAPKE $\rightarrow$ ML-KEM	Fujisaki–Okamoto transform: re-encrypt and check, or implicitly reject

10 Exercises (Assign Before Lecture 4)

Exercise: Homework Set

1. Redo the compression worked example (Section 3) with $d = 3$ instead of $d = 2$ (still $q = 17$). Is the recovered value after decompression closer to the original 9 than it was with $d = 2$? Explain why, in general, larger d means smaller compression error.
2. In the noise-cancellation derivation (Section 5), identify by name every term that ends up inside ε. Which of these terms come from key generation, and which come from encryption?
3. Using the worked example in Section 6, redo encryption with a *different* message $m = (0, 1, 0, 1)$, keeping the same A, s, e, r, e_1, e_2. Compute u, v, and confirm decryption recovers $(0, 1, 0, 1)$.
4. In your own words (4–5 sentences): why is it not enough for ML-KEM's Decaps to simply "return an error" when the re-encryption check fails, instead of the implicit-rejection mechanism in Section 7? Think about what an attacker could learn from the mere presence or timing of an error message.
5. **Looking ahead:** skim the abstract of one published Kyber side-channel paper of your choosing (we will assign specific ones in Session 5) and identify which single line of the algorithms in Section 4 of this lecture it targets — KeyGen, Enc, Dec, or the FO check in Section 7.